

One-to-one SkewGRAM : from biology to NLP

Bechir Mady¹, Abderahmane Abderrahmane¹, Mohamedou Yahya Cheikh
Mohamed Vall¹, El Moustapha Mohamed El Moustapha¹, Mohamed Lemine
Ahmed Sidi¹, and Hafedh Mohamed Babou²

¹ ¹Unité de Recherche en Systèmes Intelligents Avancés (URSIA), Institut Supérieur
du Numérique (SupNum), Nouakchott, Mauritanie

{22014, 22053, 25239, 22013}@supnum.mr, m-lamine.ahmed-sidi@supnum.mr

² ²École Supérieure Polytechnique (ESP), Nouakchott, Mauritanie
hafedh.mohamed-babou@esp.mr

Résumé Le problème One-to-one SkewGRAM est un défi d’optimisation combinatoire NP-difficile issu de la bio-informatique. Étant donné un graphe orienté acyclique D et un graphe non-orienté G définis sur le même ensemble d’entités biologiques, l’objectif est de trouver le plus long chemin dans D dont les sommets induisent un sous-graphe connexe dans G . Les méthodes d’optimisation exactes peuvent fournir des solutions optimales mais deviennent extrêmement coûteuses en temps de calcul, tandis que les approches récentes par Réseaux de Neurones sur Graphes ont tendance à générer des chemins très courts.

Nous proposons **One-to-one SkewGRAM**, une approche d’embeddings de nœuds de type Skip-Gram basée sur des marches aléatoires orientées sur D . Les représentations apprises sont combinées avec un décodeur guidé intégrant trois signaux complémentaires : une anticipation exacte par programmation dynamique sur D , la similarité d’embeddings, et un bonus de connexité basé sur G . Puisque ces signaux guident la recherche sans imposer la contrainte globale de connexité, une procédure finale exacte à fenêtre glissante basée sur l’algorithme Union-Find extrait le plus long sous-chemin contigu satisfaisant l’exigence de connexité.

L’approche proposée combine ainsi l’apprentissage de représentations avec des algorithmes de graphes exacts : les embeddings guident l’exploration, la programmation dynamique préserve le potentiel du chemin orienté, et l’étape d’extraction finale garantit la connexité requise. Une évaluation expérimentale sur des instances synthétiques est utilisée pour évaluer la qualité des solutions et l’efficacité calculatoire face aux approches exactes et d’apprentissage.

Keywords: Optimisation Combinatoire · Bio-informatique · Algorithmes sur Graphes · Embeddings de Nœuds · Skip-Gram · Programmation Dynamique · Composantes Connexes · Union-Find.

1 Introduction

L’intelligence artificielle (IA) et l’apprentissage automatique connaissent une adoption croissante en bio-informatique pour analyser des données biologiques

complexes. Les systèmes biologiques peuvent naturellement être représentés sous forme de réseaux, où les sommets correspondent aux entités biologiques et les arêtes représentent leurs relations. Combiner des structures de réseaux complémentaires est particulièrement utile pour découvrir des motifs fonctionnels conservés.

Par exemple, le métabolisme cellulaire peut être représenté par un graphe orienté, où les métabolites sont reliés par des réactions biochimiques dirigées illustrant un flux causal. Simultanément, la proximité génomique ou les interactions protéine-protéine (PPI) [7] peuvent être modélisées par un graphe non-orienté décrivant des associations fonctionnelles. L'identification de structures simultanément significatives dans ces deux graphes peut ainsi révéler d'importants motifs biologiques.

Le problème **One-to-one SkewGRAM** formalise cette tâche. Étant donnés un graphe orienté acyclique D et un graphe non-orienté G sur le même ensemble de sommets, l'objectif est de trouver le plus long chemin orienté dans D dont les sommets induisent un sous-graphe connexe dans G . Ce problème est NP-difficile [1,?].

La difficulté provient de l'interaction entre deux contraintes structurelles différentes. Un successeur valide dans D ne préserve pas nécessairement la connexité dans G , tandis que des sommets fonctionnellement connectés dans G peuvent ne pas former un chemin orienté dans D .

Des travaux récents ont exploré des approches par apprentissage profond pour ce problème [2]. En particulier, une architecture GCN double avec post-traitement heuristique et un encodeur GCN combiné à un décodeur LSTM-Attention (DL2) ont été étudiés. Bien que ces approches démontrent le potentiel des méthodes basées sur l'apprentissage, la stratégie de décodage séquentiel génère des chemins très courts, DL2 produisant environ 1,2 nœud en moyenne. À l'inverse, le solveur exact ILP2 [1] obtient des solutions optimales mais nécessite environ 28 secondes par instance de 100 nœuds.

Ces limites motivent une approche hybride combinant des représentations apprises avec des algorithmes de graphes explicites. Notre idée principale est de réinterpréter les marches aléatoires orientées sur D comme des phrases, suivant le paradigme Skip-Gram et DeepWalk/node2vec [4,?,?]. Les nœuds jouent le rôle de mots, tandis que les nœuds apparaissant dans des contextes similaires reçoivent des représentations liées.

Les embeddings appris seuls, cependant, ne suffisent pas pour résoudre le problème d'optimisation complet. Nous les utilisons donc comme signal de guidage au sein d'un décodeur spécifique au problème. Le décodeur combine la similarité des embeddings avec une anticipation exacte par programmation dynamique sur D et un signal de connexité souple issu de G . Une étape d'extraction exacte finale vérifie la contrainte de connexité.

Les contributions de cet article sont :

1. Un modèle d'embeddings de type Skip-Gram pour les nœuds d'un DAG, utilisant des marches aléatoires orientées, un sous-échantillonnage des hubs, et un échantillonnage négatif structurel.

2. Un décodeur guidé combinant une anticipation exacte du plus long chemin dans le DAG, la similarité d’embeddings apprise, et un bonus de connexité à G .
3. Une procédure exacte Union-Find à fenêtre glissante qui extrait le plus long sous-chemin (D, G) -consistant à partir de chaque candidat généré.
4. Une évaluation empirique montrant un solide compromis qualité/temps comparé au solveur exact ILP2 et à l’approche GNN précédemment rapportée.

2 Formulation du Problème et Propriété Mathématique

Soit

$$V = \{v_1, \dots, v_n\}$$

un ensemble d’entités biologiques. Nous considérons deux graphes définis sur V :

- $D = (V, A)$: un graphe orienté acyclique modélisant une relation causale ou séquentielle ;
- $G = (V, E)$: un graphe non-orienté modélisant des associations fonctionnelles.

L’objectif est de trouver un chemin

$$P = (p_1, \dots, p_k)$$

maximisant son nombre de sommets, sous les conditions :

$$(p_i, p_{i+1}) \in A, \quad i = 1, \dots, k-1, \tag{1}$$

et

$$G[\{p_1, \dots, p_k\}] \text{ est connexe.} \tag{2}$$

Le chemin est par conséquent valide dans D tandis que l’ensemble de ses sommets est connexe dans G . Nous appelons un tel chemin un **chemin** (D, G) -**consistant**.

2.1 La Connexité ne Peut pas Être Vérifiée Incrémentalement

L’exigence de connexité est globale et ne peut généralement pas être réduite à un test local entre nœuds consécutifs. En particulier, une solution valide peut contenir des sommets qui ne sont pas adjacents dans G à leur prédécesseur immédiat mais qui sont connectés à l’ensemble sélectionné via d’autres sommets.

Un audit empirique des solutions optimales ILP2 montre également que les préfixes intermédiaires ne fournissent généralement pas une base fiable pour imposer la condition de connexité finale. Par conséquent, rejeter simplement

D et G partagent les mêmes nœuds mais pas la même structure

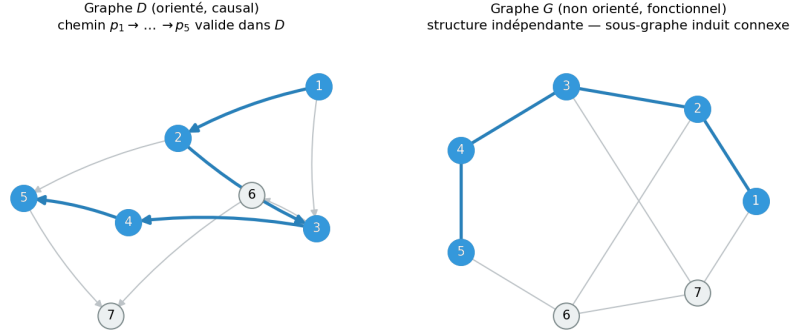


FIGURE 1. Exemple jouet sur sept nœuds. D et G partagent le même ensemble de nœuds mais représentent des relations différentes. Une solution valide doit être un chemin orienté dans D tandis que ses sommets induisent un sous-graphe connexe dans G .

chaque candidat qui devient temporairement déconnecté dans G peut éliminer des solutions valides.

Notre décodeur utilise par conséquent G uniquement comme signal de guidage souple pendant la génération. La contrainte de connexité finale est vérifiée exactement après qu'un chemin candidat orienté a été généré.

3 Méthodologie : One-to-one Skew-GRAM

Notre approche comporte deux composantes principales : un modèle d'embeddings entraîné sur des marches aléatoires orientées sur D , et un décodeur guidé qui combine les représentations apprises avec les informations structurelles de D et G .

La procédure complète comporte quatre phases :

1. génération de marches aléatoires et sous-échantillonnage des hubs ;
2. apprentissage d'embeddings de type Skip-Gram ;
3. décodage guidé conscient du DAG ;
4. extraction exacte du sous-chemin (D, G) -consistant.

3.1 Démarche générale : de l'instance au chemin, via Encoder et Decoder

L'analogie centrale est :

nœud $\leftrightarrow$ mot, marche aléatoire $\leftrightarrow$ phrase, contexte $\leftrightarrow$ voisinage dans une marche.

Les marches aléatoires sont générées uniquement sur D , en suivant les arcs sortants. Cela garantit que le corpus appris reflète la structure directionnelle requise par le chemin final.

Une fenêtre glissante est ensuite utilisée pour générer des paires centre-contexte. Les nœuds à haute fréquence sont sous-échantillonnés pour réduire l'influence des hubs.

Les paires résultantes sont utilisées pour entraîner deux matrices d'embeddings en utilisant un échantillonnage négatif. Contrairement au Skip-Gram standard, les exemples négatifs sont sélectionnés structurellement parmi les non-successeurs dans D .

Pendant le décodage, les embeddings appris sont combinés avec une anticipation exacte par programmation dynamique et un bonus de connectivité souple G . Le décodeur est exécuté de manière répétée à partir de différents nœuds de départ, et le meilleur candidat est conservé.

Enfin, parce que les signaux de guidage ne garantissent pas la connectivité globale, une procédure exacte à fenêtre glissante Union-Find extrait le plus long sous-chemin contigu qui est connexe dans G .

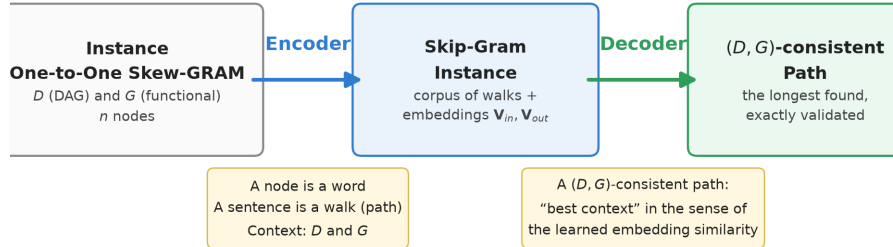


FIGURE 2. Démarche générale de One-to-one Skew-GRAM. L'encodeur transforme l'instance en un problème d'apprentissage de type Skip-Gram, tandis que le décodeur utilise les embeddings appris et les informations spécifiques aux graphes pour générer et valider des chemins candidats.

3.2 Phase 1 : Marches Aléatoires et Sous-échantillonnage des Hubs sur D

Les marches aléatoires sont générées en suivant uniquement les arcs sortants de D . Pour un nœud courant u , le nœud suivant est échantillonné parmi son

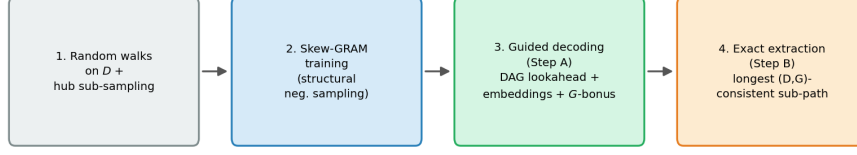


FIGURE 3. Pipeline One-to-one Skew-GRAM en quatre phases, exécuté une fois par instance.

ensemble de successeurs $N^+(u)$. La marche se termine lorsqu'un puits est atteint ou lorsque la longueur maximale de marche est atteinte.

Les marches résultantes sont interprétées comme des phrases. Une fenêtre de contexte est ensuite appliquée pour générer des paires centre-contexte.

Les nœuds à haute fréquence sont sous-pondérés en utilisant la stratégie de sous-échantillonnage standard de word2vec. Cela réduit la domination des hubs lors de l'apprentissage des représentations sans modifier le graphe utilisé par le décodeur.

3.3 Phase 2 : Modèle d'Embeddings One-to-one Skew-GRAM

Deux matrices d'embeddings

$$\mathbf{V}_{in}, \mathbf{V}_{out} \in \mathbb{R}^{N \times d}$$

sont entraînées, où $N = |V|$ et $d = 32$.

Pour un nœud centre c , un nœud contexte positif o , et K échantillons négatifs $n_1, \dots, n_K$, la perte d'échantillonnage négatif est

$$\mathcal{L}(c, o, n_{1:K}) = -\log \sigma(\mathbf{v}_{in,c}^\top \mathbf{v}_{out,o}) - \sum_{k=1}^K \log \sigma(-\mathbf{v}_{in,c}^\top \mathbf{v}_{out,n_k}). \quad (3)$$

La principale modification par rapport à l'objectif Skip-Gram standard est l'**échantillonnage négatif structurel**. Les nœuds négatifs sont échantillonnés parmi les nœuds qui ne sont pas des successeurs du nœud centre dans D .

Cela rend le signal négatif sensible à la structure directionnelle du problème plutôt que de s'appuyer uniquement sur la fréquence.

La similarité utilisée lors du décodage est

$$\text{sim}(u, v) = \mathbf{v}_{in,u}^\top \mathbf{v}_{out,v}. \quad (4)$$

Le modèle est entraîné indépendamment pour chaque instance car les identifiants de nœuds n'ont pas de signification sémantique commune à travers les graphes générés indépendamment.

3.4 Phase 3 : Décodage Guidé Conscient du DAG (Étape A)

Un décodeur basé uniquement sur la similarité d'embeddings peut sélectionner des successeurs localement attractifs qui mènent rapidement à des puits. Pour réduire ce comportement, nous calculons une anticipation exacte du plus long chemin sur D .

Pour chaque nœud v , on définit

$$\text{lf}(v) = \begin{cases} 0, & N^+(v) = \emptyset, \\ \max_{w \in N^+(v)} (1 + \text{lf}(w)), & \text{sinon.} \end{cases} \quad (5)$$

Puisque D est un DAG, toutes les valeurs $\text{lf}(v)$ sont calculées en une seule passe topologique inversée en

$$O(|V| + |A|).$$

À chaque étape de décodage, soit u le nœud courant et $v \in N^+(u)$ un successeur candidat. Nous définissons

$$S(u, v) = \frac{\text{lf}(v)}{\tau_{lf}} + \beta \frac{\text{sim}(u, v)}{\tau} + \log(\gamma) \mathbb{I}[v \text{ est adjacent dans } G \text{ à un nœud sélectionné}]. \quad (6)$$

Le nœud suivant est échantillonné selon

$$P(v \mid u) = \frac{\exp(S(u, v))}{\sum_{w \in N^+(u)} \exp(S(u, w))}. \quad (7)$$

Les trois termes représentent respectivement :

1. le potentiel de chemin orienté restant dans D ;
2. la compatibilité d'embeddings apprise ;
3. une préférence souple pour les candidats connectés dans G au chemin construit jusqu'à présent.

Les nœuds déjà visités sont masqués pour préserver la contrainte one-to-one. Le décodeur est répété 30 000 fois par instance en utilisant différents points de départ, et le meilleur chemin extrait final est conservé.

3.5 Phase 4 : Extraction Exacte du Sous-chemin (D, G) -Consistant (Étape B)

Le candidat généré à la Phase 3 est garanti d'être un chemin orienté dans D , mais il n'est pas nécessairement connexe dans G .

Pour un chemin candidat

$$P = (p_1, \dots, p_L),$$

nous recherchons donc le plus long intervalle contigu $[i, j]$ tel que

$$G[\{p_i, \dots, p_j\}]$$

soit connexe.

Un sous-chemin contigu d'un chemin valide dans D reste un chemin valide dans D . Ainsi, cette dernière étape n'a besoin d'imposer que la condition de connexité G .

Nous utilisons une procédure de fenêtre glissante combinée à une structure Union-Find incrémentale [9]. Pour un chemin candidat de longueur L , la complexité dans le pire des cas est

$$O(L^2\alpha(L)),$$

où α est la fonction inverse d'Ackermann.

Cette étape finale est exacte : chaque chemin retourné par l'algorithme satisfait la définition d'un chemin (D, G) -consistant.

4 Résultats et Discussion

4.1 Configuration Expérimentale

Nous évaluons la méthode proposée sur 38 instances synthétiques d'Erdős–Rényi [10], contenant chacune 100 nœuds. Un optimum ILP2 est disponible pour 36 des 38 instances.

La comparaison inclut :

- GNN DL2 [2] ;
- ILP2 [1] ;
- One-to-one Skew-GRAM.

La dimension des embeddings est 32, avec 40 marches par nœud de départ, une longueur de marche maximale de 30, une fenêtre de contexte de 3, et cinq échantillons négatifs structurels par paire positive. Le décodeur effectue 30 000 marches Monte-Carlo par instance.

Toutes les expériences utilisent la graine aléatoire fixe `SEED=42`. La configuration complète des hyperparamètres est fournie en Annexe A.

TABLE 1. Comparaison des méthodes sur 38 instances synthétiques de 100 nœuds. Les optima ILP2 sont disponibles pour 36 instances.

Méthode	Longueur Moyenne	Temps Moyen	Accélération
GNN DL2 [2]	≈ 1.2	N/A	–
ILP2 [1]	9.97	28.0 s	référence
One-to-one Skew-GRAM	7.13	2.7 s	$\sim 10\times$

La méthode proposée obtient une longueur de chemin moyenne de 7,13 nœuds, comparée à 9,97 nœuds pour ILP2. La différence relative moyenne correspondante est d'environ

$$\frac{7.13 - 9.97}{9.97} \approx -28.5\%.$$

Le temps d'exécution moyen diminue de 28,0 secondes pour ILP2 à 2,7 secondes pour One-to-one Skew-GRAM, correspondant à une accélération d'environ 10,4×.

Les résultats indiquent donc un compromis qualité/temps substantiel : l'heuristique proposée ne retrouve pas systématiquement l'optimum exact, mais elle obtient des chemins considérablement plus longs que l'approche GNN précédente tout en nécessitant beaucoup moins de calculs qu'ILP2.

TABLE 2. Comparaison instance par instance entre One-to-one Skew-GRAM et ILP2 sur dix instances représentatives de 100 nœuds.

Instance	ILP2 Skew-GRAM	Écart Temps ILP2 Temps Skew-GRAM
(nœuds)	(nœuds)	(%) (s) (s)
100_121	6	6 0.0% 19.59 0.69
100_402	11	11 0.0% 13.16 0.80
100_190	11	11 0.0% 25.44 0.77
100_197	11	9 -18.2% 19.73 0.70
100_14	12	9 -25.0% 8.51 0.77
100_320	12	8 -33.3% 33.91 0.80
100_128	13	8 -38.5% 18.44 0.90
100_66	16	9 -43.8% 9.98 0.97
100_388	15	8 -46.7% 58.18 0.84
100_527	11	4 -63.6% 15.83 1.04

Le Tableau 2 illustre la variabilité de l'heuristique selon les instances. One-to-one Skew-GRAM correspond exactement à ILP2 sur plusieurs instances, dont 100_121, 100_402 et 100_190. Sur les 36 instances avec un optimum connu, il atteint l'optimum ILP2 sur 13 instances.

Le temps d'exécution reste considérablement plus faible pour toutes les instances présentées. La différence est particulièrement visible pour des instances telles que 100_388, où ILP2 nécessite 58,18 secondes alors que la méthode proposée ne nécessite que 0,84 seconde.

La variabilité de la qualité de la solution est attendue car le décodeur repose sur un échantillonnage stochastique. Certaines instances contiennent des chemins prometteurs qui sont plus faciles à atteindre grâce au guidage appris, tandis que d'autres nécessitent l'exploration de branches moins probables.

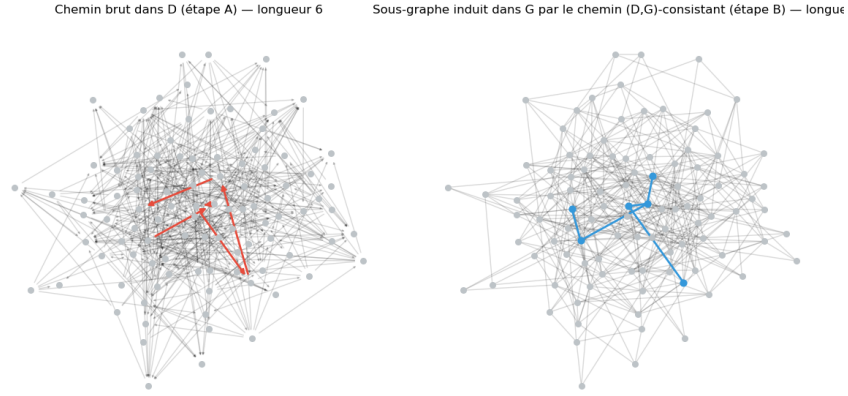


FIGURE 4. Exemple du processus de décodage en deux étapes. L'Étape A génère un chemin orienté dans D , tandis que l'Étape B extrait le plus long sous-chemin contigu qui est connexe dans G .

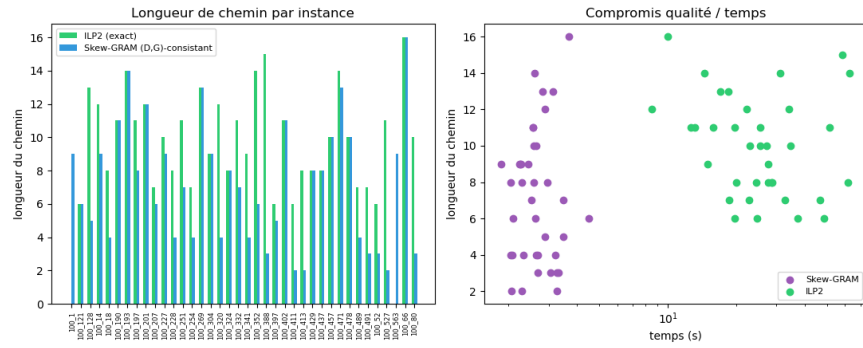


FIGURE 5. Comparaison de la longueur de chemin par instance entre One-to-one Skew-GRAM et ILP2, ainsi que le compromis qualité-temps sur l'ensemble des instances évaluées.

4.2 Ablation de l’Anticipation DAG

Nous évaluons également l’importance de l’anticipation DAG exacte en la retirant du décodeur.

Sans l’anticipation, le décodage basé principalement sur la similarité d’embeddings produit une longueur de chemin moyenne de 5,87 nœuds, comparée à 7,13 nœuds pour l’approche complète.

Cela représente une réduction d’environ 17,7%.

Ce résultat confirme que la similarité d’embeddings seule ne capture pas adéquatement le potentiel de chemin restant dans le DAG. Un nœud candidat peut être localement compatible avec le nœud courant tout en menant à une branche courte ou à un puits.

L’anticipation par programmation dynamique fournit cette information explicitement et aide donc le décodeur à éviter les culs-de-sac prématurés.

4.3 Discussion

Les résultats soulignent l’avantage de combiner des composantes apprises et algorithmiques.

Premièrement, la grande différence entre DL2 et One-to-one Skew-GRAM suggère que le décodage neuronal purement séquentiel a des difficultés à maintenir des chemins suffisamment longs sous les contraintes couplées de D et G .

Deuxièmement, l’anticipation DAG fournit une information exacte qui peut être calculée efficacement car D est acyclique. Plutôt que d’exiger du modèle d’embeddings qu’il apprenne cette propriété implicitement, nous la fournissons directement au décodeur.

Troisièmement, G est délibérément utilisé comme structure de guidage et de validation plutôt que d’être incorporé dans l’objectif de l’embedding. Cela permet de garder le problème d’apprentissage de représentation concentré sur la structure dirigée qui définit l’ordre de chemin valide, tandis que la contrainte de connexité globale est gérée explicitement pendant le décodage et l’extraction.

La méthode proposée ne doit donc pas être interprétée comme un remplacement d’ILP2 lorsque l’optimalité garantie est requise. Son principal avantage est le compromis qualité-temps : elle obtient des chemins considérablement plus longs que l’approche GNN précédente tout en étant environ dix fois plus rapide que le solveur exact sur les instances évaluées.

5 Conclusion

Nous avons présenté **One-to-one SkewGRAM**, une approche hybride pour le problème NP-difficile One-to-one SkewGRAM. La méthode combine des embeddings de nœuds de type Skip-Gram appris à partir de marches aléatoires orientées sur D avec une anticipation DAG exacte, une stratégie de décodage guidée par la connexité, et une étape finale d’extraction exacte basée sur Union-Find.

Les résultats expérimentaux sur 38 instances synthétiques de 100 nœuds montrent que l’approche proposée offre un solide compromis qualité/temps. Elle obtient une longueur de chemin moyenne de 7,13 nœuds en 2,7 secondes, contre 9,97 nœuds en 28,0 secondes pour le solveur exact ILP2. Elle atteint l’optimum ILP2 sur 13 des 36 instances pour lesquelles un optimum est disponible, tout en nécessitant beaucoup moins de calcul. L’étude d’ablation confirme également l’importance de l’anticipation DAG pour maintenir des chemins candidats plus longs.

Ces résultats suggèrent que la combinaison de représentations de nœuds apprises avec des algorithmes de graphes spécifiques au problème peut constituer une alternative efficace à la fois à l’optimisation purement exacte et au décodage neuronal de bout en bout pour les problèmes combinatoires sur graphes couplés.

A Hyperparamètres

Le Tableau 3 donne les hyperparamètres utilisés pour obtenir les résultats rapportés.

TABLE 3. Hyperparamètres utilisés par l’encodeur et le décodeur.

Paramètre	Symbole	Valeur	Rôle
<i>Encoder – Phases 1–2</i>			
Marches par sommet de départ	–	40	Taille du corpus de marches sur D
Longueur maximale d’une marche	–	30	Arrêt anticipé sur un puits
Fenêtre de contexte	–	3	Génération de paires centre-contexte
Dimension des embeddings	d	32	Taille des embeddings de nœuds
Époques d’entraînement	–	8	Optimiseur Adam [16]
Négatifs structurels par paire	K	5	Échantillons négatifs
Taux d’apprentissage	–	0.01	Taux d’apprentissage Adam [16]
<i>Decoder – Phases 3–4</i>			
Marches Monte-Carlo au décodage	–	30 000	Nombre de chemins candidats
Temp. du signal d’embedding	τ	0.5	Température du score d’embedding
Poids du signal d’embedding	β	0.3	Poids de la similarité d’embeddings
Bonus de connexité à G	γ	6.0	Préférence de connexité
Temp. de l’anticipation DAG	τ_{lf}	1.5	Température du score d’anticipation

Références

1. Ahmed Sidi, M. L., Babou, H. M., Dhib, C., Neron, E., Soukhal, A., Nanne, M. F. : Constraint programming approaches for finding conserved metabolic and genomic patterns. *Computers & Operations Research*, 183 :107166 (2025)
2. Sidi, M. L. A., Babou, H. M., Dhib, C. : Deep Learning approaches for finding a longest DG-consistent path in biological networks. Working paper / Preprint (2024)
3. Zaharia, A., et al. : CoMetGeNe : Mining conserved neighborhood patterns in metabolic and genomic contexts. *BMC Bioinformatics*, 20(1) :1–20 (2019)
4. Mikolov, T., Sutskever, I., Chen, K., Corrado, R., Dean, J. : Distributed Representations of Words and Phrases and their Compositionality. In : *Advances in Neural Information Processing Systems* (2013)
5. Perozzi, B., Al-Rfou, R., Skiena, S. : DeepWalk : Online Learning of Social Representations. In : *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2014)
6. Grover, A., Leskovec, J. : node2vec : Scalable Feature Learning for Networks. In : *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2016)
7. Sharan, R., Ideker, T. : Modeling protein interaction networks. *Nature Reviews Genetics*, 7(7) :575–582 (2006)
8. Fertin, G., Babou, H. M., Rusu, I. : Algorithms for sub-network mining in heterogeneous networks. *Lecture Notes in Computer Science*, 7276 :184–194 (2012)
9. Tarjan, R. E. : Efficiency of a Good But Not Linear Set Union Algorithm. *Journal of the ACM*, 22(2) :215–225 (1975)
10. Erdős, P., Rényi, A. : On Random Graphs I. *Publicationes Mathematicae*, 6 :290–297 (1959)
11. Bellman, R. : On a Routing Problem. *Quarterly of Applied Mathematics*, 16(1) :87–90 (1958)
12. Vinyals, O., Fortunato, M., Jaitly, N. : Pointer networks. *Advances in Neural Information Processing Systems*, 28 (2015)
13. Bello, I., Pham, H., Le, Q. V., Norouzi, M., Bengio, S. : Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv :1611.09940* (2016)
14. Dai, H., Khalil, E. B., Zhang, Y., Dilkina, B., Song, L. : Learning combinatorial optimization algorithms over graphs. *Advances in Neural Information Processing Systems*, 30 (2017)
15. Karalias, K., Loukas, A. : Erdős goes neural : an unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems*, 33 :3135–3145 (2020)
16. Kingma, D. P., Ba, J. : Adam : A method for stochastic optimization. *arXiv preprint arXiv :1412.6980* (2014)